

Course Scheduling Project Report

Jay Lee, Jehoon Park, Sanghwi Park

May 3, 2026

Abstract

The course scheduling problem is a problem that grows exponentially based on the number of courses. In this project, we model this problem as a Constraint Satisfaction Problem (CSP). We implemented a basic backtracking algorithm, heuristic-based search utilizing the Minimum Remaining Values (MRV) and degree heuristics, and a constraint propagation technique using Maintaining Arc Consistency (MAC).

We evaluate and compare each algorithm based on the pre-created datasets(set of courses and sections) by analyzing runtime, count of visited nodes, and backtracking.

Our experiment results show that for relatively small or simple datasets, basic backtracking performs better in terms of execution time than heuristic-based and constraint propagation approaches due to lower computational overhead. However, when the dataset grows and constraints become more complex, the algorithm incorporating heuristic and constraint propagation (MAC) showed better performance by reducing the search space.

1 Introduction

The process of creating a class schedule for a university student entails selecting a combination of courses that satisfies various constraints. Each course may have multiple sections, and sections offered at different times. A student must choose sections that do not conflict with one another.

This is a computationally challenging problem because the number of available combinations of courses will grow rapidly as the number of courses increases. This problem can be naturally modeled as a Constraint Satisfaction Problem (CSP), as described by Barták et al. [1].

According to Edward Tsang’s book on CSP, CSP is defined by a set of variables, domains, and constraints [9].

- Variable: Selection of Individual Subjects or Sections
- Domain: Possible sections for each subject
- Constraint: No time conflicts between the selected classes

In other words, the goal is to find a combination of classes that satisfies all constraints.

We can solve problems in a structured way using the Constraint Satisfaction Problem (CSP), which provides a foundation for systematically exploring possible solutions. It is effective for problems that have a bunch of constraints, such as course scheduling. As described in prior work, CSP provides a structured framework for systematically exploring possible solutions and has been widely applied to problems with numerous constraints, such as scheduling [1].

To address the problem of course scheduling, we implement various search strategies based on Constraint Satisfaction Problems (CSPs). We begin with a basic backtracking algorithm, and we subsequently refine by incorporating techniques, such as the Minimum Remaining Values (MRV) and Degree heuristics, to enhance the efficiency of variable selection. Furthermore, we integrate constraint propagation techniques utilizing MAC (Maintaining Arc Consistency) to enable more effective pruning of the search space.

In evaluating these approaches, we compare and analyze the performance of each algorithm based on metrics such as execution time(runtime), the number of visited nodes, and the count of backtracking operations. Through this comparative analysis, we aim to validate the effectiveness of heuristic and constraint propagation techniques in solving the course scheduling problem.

2 Background

2.1 Backtracking

Backtracking is one of the fundamental search algorithms for solving Constraint Satisfaction Problems (CSP) [1]. According to Bitner and Reingold [2], the backtracking algorithm incrementally extends a solution by assigning values to variables one by one, and if a constraint is violated, it goes back to the previous step and makes a different choice. This helps the systematic exploration of all possible solutions.

The operation process for the backtracking algorithm is as follows:

- A variable is selected, and a value is assigned to it from the domain.
- Verified whether the constraints are satisfied, and proceeds to the next variable.

- Otherwise, backtracks to the previous step to attempt a different value.
- By repeating this process, all possible cases are explored [2].

Because a backtracking algorithm explores all possible cases, it can guarantee finding a solution if one exists. However, this completeness simultaneously leads to a major drawback. Exploring all possible cases implies that, in the worst case, the time complexity increases exponentially. Consequently, for problems involving variables and constraints such as course scheduling, the search space becomes extremely large, which can lead to inefficiency [1]. To address this limitation, some techniques like heuristics and constraint propagation have been proposed.

2.2 Heuristics

To address this challenge, the choice of efficient heuristics becomes essential for reducing the search space and solving the problem within a reasonable time.

Haralick and Elliot mention applying such heuristics based on the fail-first principle, combined with lookahead techniques like forward checking, to produce a better performance compared to the standard backtracking search. According to their paper, the fail-first heuristic chooses the variable most likely to fail the consistency check, so they don't have to run the remaining tests for the others if it fails. Their choice of fail-first heuristics to "optimize" the order of consistency tests for the inference procedure indeed reduced the number of checks, and this trend was far enhanced for larger problem size(i.e., Number of variables) [6].

The minimum-remaining-values (MRV), or saturation degree (SD), heuristic is one type of fail-first heuristic often applied to course scheduling problems to prune the search tree more effectively by selecting a variable with the fewest remaining legal values when assigning a value. The MRV(SD) heuristic tends to perform better for the scheduling problems compared to other heuristics, such as random ordering and the degree heuristic(LD) in general [4]. The observations from Carter and Larporte's paper support the efficiency of MRV heuristics, especially as the problem becomes more complex. The study compared 5 different heuristics, including MRV, RO, LD, and two others, to solve a set of both random and real examination scheduling problems in terms of speed and answer quality. Among the five heuristics, MRV overall yielded the best performance in execution time, while LD produced a better solution quality most of the time. Still, MRV performed better in all measurements for all three 'difficult' problems identified in the paper, compared to other heuristics covered, showing its fitness for scheduling problems [4].

The major downside of the MRV heuristic is its inability to select a variable when a tie appears among unassigned variables. To resolve this problem, when MRV is unavailable, the degree heuristic selects the variable involved in the largest number of constraints with other variables, which is more likely to lead to conflicting results in subsequent steps and fail early.

While heuristics can provide an optimal searching order, they can't prevent the algorithm from wasting time wandering around at branches that cannot lead to a valid solution aimlessly, just to fail eventually. For this reason, heuristics are often paired with "lookahead" algorithms, which can detect conflicts early in the searching branch by employing constraint propagation techniques. Therefore, this project plans to apply MRV and Degree heuristics to improve variable selection and efficiently reduce the search space.

2.3 Propagation

Constraint propagation is a technique that can be applied before and during the search to narrow down the possible domain of each variable, to prune branches early that are guaranteed to fail, and therefore to reduce the size of the search space. In backtracking search, it is implemented through consistency inference algorithms like forward checking and the MAC algorithm [7].

At first, it may seem like overkill to run the entire AC-3 on each node at every step rather than a partial check through forward checking, considering the additional inference cost induced by the high complexity of the AC-3 algorithm. Sabin and Freuder's study refutes such a belief by showing that maintaining full arc consistency during search is, in fact, a very cost-effective approach, as search savings produced by pruning more values often outweigh the additional costs of running the full consistency check. The study compared the runtime of FC, AC-FC, and MAC on solving a set of randomly generated CSPs with varying complexity, tightness, and density, producing the following key observations [8]:

- MAC, which employs the full arc consistency maintenance during searching, was superior to both FC and AC-PC except on some very simple problems.
- Adding partial arc consistency checking with AC preprocessing alone can sometimes decrease performance, but adding even more consistency processing by combining AC preprocessing plus full AC maintenance can increase the performance.
- The performance gap between FC and MAC grew as the constraint tightness increased, and the density decreased [8].

Similar trends apply to course scheduling problems, where constraint tightness is generally high, as many courses must fit a restrictive range of time. As shown from Sabin and Freuder's observations, forward checking cannot detect distant dead-ends effectively with a shallow search depth, constrained to immediate conflicts in immediate neighbors, and often trapped in branches that cannot lead to a valid solution, making MAC a more suitable choice for course scheduling.

The AC-3 algorithm, which serves as the basis of MAC, was originally introduced by Mackworth [7]. In his work, Mackworth identified three key sources of inefficiency in backtracking

search node inconsistency, arc inconsistency, and path inconsistency and proposed consistency algorithms as remedies to eliminate these issues before search begins. Specifically, AC-3 enforces arc consistency by maintaining a queue of arcs to be checked. For each arc (x, y) , values are removed from the domain of x for which no consistent value exists in the domain of y . If the domain of x is reduced, all arcs pointing to x are re-added to the queue, as previous consistency checks may no longer hold. This process repeats until no further reductions are possible. By systematically eliminating local inconsistencies prior to and during search, AC-3 substantially reduces the search space and prevents the repeated discovery of the same failures that makes naive backtracking inefficient [7]. Based on these research, we plan to incorporate MAC into our project to maintain arc consistency during the search process and prune impracticable branches early on.

2.4 CSP Application in Course Scheduling

Course scheduling is one of the most representative applications of CSP. As noted by Brailsford et al. [3], many combinatorial problems in operational research, such as scheduling and timetabling, can be naturally formulated as CSPs. However, Fox and Sadeh identify several key characteristics that make scheduling particularly difficult from a CSP perspective, including its enormous search space, constraints, and the possibility that no solution exists [5]. To address these challenges, our approach combines backtracking with MRV and degree heuristics, along with MAC. As demonstrated by Brailsford et al. [3], constraint satisfaction approaches using tree search combined with backtracking and consistency checking are well-suited for scheduling problems, and MAC's ability to detect conflicts early and aggressively prune domains makes it particularly effective for course scheduling where many courses must fit within a restrictive range of available time slots.

Based on this background, our project applies backtracking, heuristic search, and constraint propagation techniques to address the course scheduling problem and evaluate their performance.

3 Approach

3.1 Problem Representation

We have modeled the course scheduling problem as a Constraint Satisfaction Problem(CSP). Each course is a variable, and all possible sections of the courses are the domain. Each section contains the meeting time and date (Mon - Fri).

Choosing exactly one section per course is going to be a solution. The most important constraint is that selected sections cannot overlap in time. This constraint will be implemented as a function that checks whether a candidate section conflicts with any previously assigned sections.

3.2 Backtracking

We implemented a recursive backtracking algorithm to explore possible assignments. At each step, the algorithm selects a first unassigned course and loops through its possible sections. For each candidate (section), it checks whether the assignments do not conflict with previously assigned sections. If there is no conflict, the assignment is extended recursively.

If all the sections cause conflicts with the current set of assigned courses, the algorithm will backtrack to the previous assignment and try alternative options. To evaluate the performance, we track the number of visited nodes and backtracking.

Algorithm 1 Basic Backtracking for Course Scheduling

```
1: function BACKTRACK(assignment)
2:   if all courses are assigned then
3:     return assignment
4:   end if
5:   nodes_visited  $\leftarrow$  nodes_visited + 1
6:   course  $\leftarrow$  first unassigned course in courses list
7:   for each section in course.sections do
8:     if section is not conflicting with assignment then
9:       assignment[course]  $\leftarrow$  section
10:      result  $\leftarrow$  BACKTRACK(assignment)
11:      if result  $\neq$  failure then
12:        return result
13:      end if
14:      remove assignment[course]
15:    end if
16:  end for
17:  backtracks  $\leftarrow$  backtracks + 1
18:  return failure
19: end function
```

3.3 Heuristic-Based Search

To improve efficiency, we implemented heuristic strategies for selecting the next course. We use the Minimum Remaining Values(MRV) heuristic, which chooses the course that has the fewest sections available first. This is implemented by the function that counts the number of sections that do not conflict with the selected sections.

When multiple courses have the same number of sections, one course is prioritized over others using the degree heuristic. The degree heuristic will choose the course that possibly causes more conflicts with the other non-selected courses compared to the others. These heuristics will help to reduce the search space by always selecting the most constrained variables.

Algorithm 2 Heuristic Course Selection (MRV + Degree)

```

1: function SELECTCOURSE(assignment)
2:   unassigned  $\leftarrow$  all courses not in assignment
3:   candidates  $\leftarrow$  courses with minimum number of valid sections (MRV)
4:   if size of candidates = 1 then
5:     return that course
6:   end if
7:   return course in candidates with maximum number of conflicts (Degree)
8: end function

```

3.4 Constraint Propagation

We implemented MAC(Maintaining Arc Consistency) for better efficiency. During the search, it maintains a domain for each course and applies the AC-3 algorithm.

The AC-3 algorithm manages the arcs between the courses as a queue and eliminates inconsistent sections from domains. And also, the section that conflicts with all the sections from other courses is also going to be eliminated. When the domain from some course is empty, it means failure, so it backtracks immediately.

For correctness, we create a copy of the domains at each recursive step before constraint propagation. This makes the domain change not affect other branches of the search tree. By pruning inconsistent values at an early stage, MAC reduces the amount of unnecessary exploration of doomed branches.

Algorithm 3 AC-3 for Maintaining Arc Consistency

```
1: function AC3(domains)
2:   queue  $\leftarrow$  all pairs of courses  $(X, Y)$  where  $X \neq Y$ 
3:   while queue is not empty do
4:      $(X, Y) \leftarrow$  pop from queue
5:     if REVISE( $X, Y, \text{domains}$ ) then
6:       if domain of  $X$  is empty then
7:         return failure
8:       end if
9:       for each course  $Z \neq X, Y$  do
10:        add  $(Z, X)$  to queue
11:      end for
12:    end if
13:  end while
14:  return success
15: end function

16: function REVISE( $X, Y, \text{domains}$ )
17:   revised  $\leftarrow$  false
18:   for each value  $x$  in domain of  $X$  do
19:     if no value  $y$  in domain of  $Y$  satisfies the constraint then
20:       remove  $x$  from domain of  $X$ 
21:       revised  $\leftarrow$  true
22:     end if
23:   end for
24:   return revised
25: end function
```

4 Experiment Design

4.1 Dataset

To evaluate the performance of each algorithm, we designed three datasets of varying sizes: small, medium, and large, along with one additional unsolvable dataset. Each dataset represents a course scheduling scenario in which a student must select one section per course without any time conflicts.

The small dataset consists of 6 courses, each with 2 sections offered at distinct time slots. The time slots are distributed across Monday/Wednesday and Tuesday/Thursday, with minimal overlap between courses. This dataset is designed to represent a simple scheduling scenario where conflicts are rare and a valid solution is easy to find.

Course	Section	Time Slot
Course1	S1	Mon/Wed 9:00–10:15
Course1	S2	Tue/Thu 10:30–11:45
Course2	S1	Mon/Wed 10:30–11:45
Course2	S2	Tue/Thu 9:00–10:15
Course3	S1	Mon/Wed 12:30–1:45
Course3	S2	Tue/Thu 12:30–1:45
Course4	S1	Mon/Wed 2:30–3:45
Course4	S2	Tue/Thu 2:30–3:45
Course5	S1	Tue/Thu 4:00–5:15
Course5	S2	Mon/Wed 4:00–5:15
Course6	S1	Mon/Wed/Fri 10:30–11:20
Course6	S2	Tue/Thu 5:00–6:15

Table 1: Small dataset (6 courses, 2 sections each).

The medium dataset consists of 15 courses, each with 2 sections. A subset of courses share overlapping time slots, introducing moderate constraint tightness. Four courses are designated as bottleneck courses, each having only a single available section, which forces the other courses to work around them.

Course	Section	Time Slot
Course1	S1	Mon/Wed 9:00–10:15
Course1	S2	Mon/Wed 2:30–3:45
Course2	S1	Mon/Wed 9:00–10:15
Course2	S2	Tue/Thu 2:30–3:45
Course3	S1	Mon/Wed 9:00–10:15 (<i>bottleneck</i>)
... 15 courses total ...		

Table 2: Medium dataset sample (15 courses, bottleneck courses have 1 section).

The large dataset consists of 20 courses, each with 2 to 3 sections. The degree of overlap between sections is intentionally increased compared to the medium dataset. Similar to the medium dataset, four bottleneck courses with a single section are included to further constrain the search space. This dataset is designed to stress-test each algorithm under higher constraint tightness.

Course	Section	Time Slot
Course1	S1	Mon/Wed 9:00–10:15
Course1	S2	Tue/Thu 9:00–10:15
Course1	S3	Mon/Wed 8:00–9:00
Course2	S1	Mon/Wed 9:00–10:15
Course2	S2	Tue/Thu 10:30–11:45
Course5	S1	Mon/Wed 9:00–10:15 (<i>bottleneck</i>)
... 20 courses total ...		

Table 3: Large dataset sample (20 courses, up to 3 sections each).

In addition, we included an unsolvable dataset consisting of 13 courses modeled after a realistic university course schedule. This dataset contains courses with heavily overlapping time slots such that no valid combination of sections satisfies all constraints. As noted by Fox and Sadeh [5], the possibility that no solution exists is one of the key characteristics that makes scheduling particularly difficult from a CSP perspective. This dataset was included to evaluate how efficiently each algorithm can determine that no valid schedule exists.

Course	Section	Time Slot
Internet Programming	S1	Mon/Wed 11:15–12:30
Computer Architecture	S1	Tue/Thu 11:15–12:30
Computer Architecture Lab	S1	Fri 10:10–11:00
Computer Architecture Lab	S2	Fri 11:15–12:05
Software Dev. Process	S1	Tue/Thu 9:45–11:00
... 13 courses total ...		

Table 4: Unsolvable dataset sample (13 courses, no valid solution exists).

All three solvable datasets were manually constructed to guarantee the existence of at least one valid solution.

4.2 Metrics

The following metrics were recorded for each algorithm and dataset combination:

- **Runtime:** Total time elapsed to find a solution, measured in seconds.
- **Nodes Visited:** Number of nodes expanded during the search.
- **Backtracks:** Number of times the algorithm backtracked due to a constraint violation.
- **Solution Found:** Whether the algorithm successfully found a valid schedule.

5 Results

Table 5 summarizes the experimental results across all four datasets and algorithm configurations.

Dataset	Mode	Runtime (s)	Nodes	Backtracks	Solution
Small (6)	Basic	1.81e-05	7	1	Yes
Small (6)	MRV + Degree	7.18e-05	6	0	Yes
Small (6)	MRV + Degree + MAC	1.46e-04	6	0	Yes
Medium (15)	Basic	6.10e-05	27	12	Yes
Medium (15)	MRV + Degree	6.37e-04	15	0	Yes
Medium (15)	MRV + Degree + MAC	1.69e-03	15	0	Yes
Large (20)	Basic	1.42e-03	429	409	Yes
Large (20)	MRV + Degree	1.13e-03	20	0	Yes
Large (20)	MRV + Degree + MAC	4.63e-03	20	0	Yes
Unsolvable (13)	Basic	9.71e-04	316	316	No
Unsolvable (13)	MRV + Degree	1.90e-04	5	5	No
Unsolvable (13)	MRV + Degree + MAC	2.63e-04	1	1	No

Table 5: Experimental results across all dataset sizes and algorithm configurations.

Figure 1 illustrates the runtime and backtrack counts across the solvable dataset sizes.

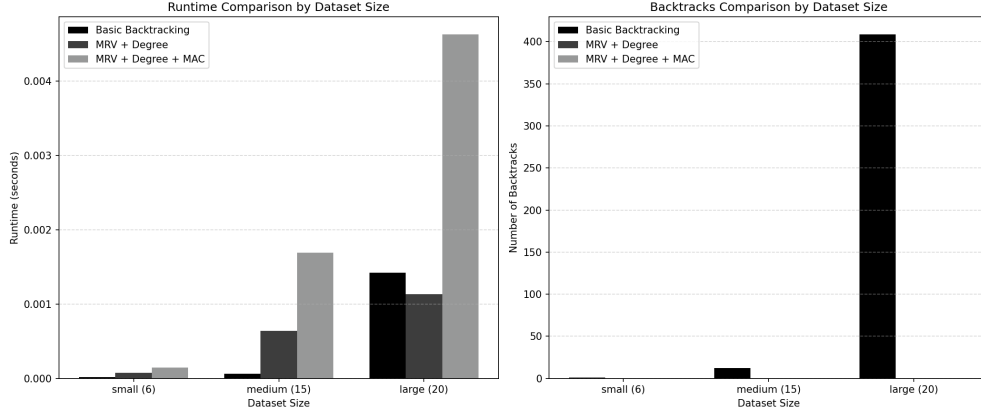


Figure 1: Runtime and backtrack comparison across dataset sizes.

Figure 2 illustrates the runtime and backtrack counts for the unsolvable dataset.

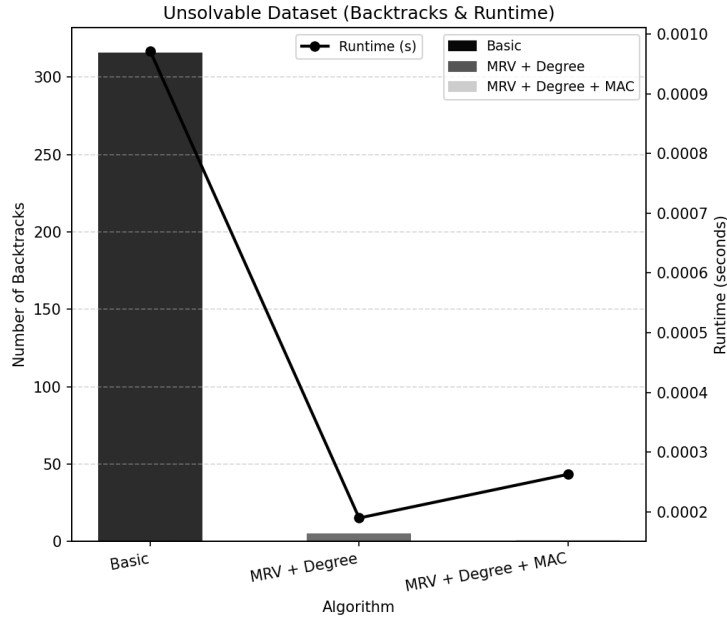


Figure 2: Runtime and backtrack comparison for the unsolvable dataset.

The main observations from the experiments are as follows:

- Across all solvable datasets, the basic backtracking algorithm consistently recorded the shortest execution time, while MAC exhibited the longest execution time, being approximately 28 times slower than the basic algorithm at worst.
- The number of visited nodes and backtracks increased at a significantly high pace from 7 in the small dataset to 409 in the large dataset for the basic backtracking algorithm

as the size of the dataset increased. In contrast, the numbers for MRV+Degree and MAC were affected only slightly by the increase in dataset size, with linear growth in visited nodes and backtrack number remaining the same.

- For the medium-size unsolvable dataset, the basic algorithm explored over 10 times the number of nodes it visited in the solvable dataset of a similar size, while the MRV+Degree and MAC had a reduced number of visited nodes from the solvable medium-size dataset.
- MRV+Degree and MAC algorithms both had the same number of backtracks and nodes visited for all solvable datasets.

6 Analysis

The experimental results reveal three key trends. First, the number of backtracks and visited nodes for basic backtracking increases sharply as the dataset grows. In contrast, both MRV+Degree and MRV+Degree+MAC consistently achieve notably smaller sizes of visited nodes and zero backtracks across all solvable dataset sizes. This confirms that fail-first heuristics and in-depth constraint propagation through MRV and MAC are effective at guiding the search toward a valid solution by early-avoiding many of the dead ends that lead to unnecessary backtracking, reducing the effective search depth of the problem.

Second, despite the reduction in search space, the runtime of MRV+Degree+MAC is the highest across all solvable dataset sizes. The high runtime ratio between the basic and MRV+Degree+MAC algorithms indicates the inefficiency of the MAC algorithm when the problem is not constrained enough so that the computational benefit from domain pruning and reduction in the search space can offset the inherent overhead of the MAC approach. Still, the performance gap in runtime between the basic and the propagation-applied algorithms reduces as the size of the dataset increases with higher tightness and lower density. This trend is consistent with Sabin and Freuder’s findings [8], which suggest that MAC outperforms simpler methods only when constraint tightness and density are sufficiently high and low enough, respectively, to justify the overhead of full arc consistency maintenance. The design of our solvable datasets further deteriorates the efficiency of MAC compared to the MRV-only approach. Since the bottleneck courses from larger solvable datasets have domains of size one from the outset, MRV can immediately identify and assign them without requiring constraint propagation, making MAC incur the cost of running AC-3 without the computational benefit expected from domain pruning. This is shown in the result table as well, where the number of nodes visited and backtracks remains the same for both MRV-only and MAC+MRV for all solvable datasets.

Third, the unsolvable dataset reveals a distinctly different pattern from the other datasets. When no valid solution exists, basic backtracking must exhaustively explore all 316 nodes before concluding that the problem is infeasible. MRV+Degree reduces this to 5 nodes, while

MAC detects infeasibility after visiting only 1 node. This demonstrates that constraint propagation implemented through MAC is effective at identifying dead ends early, proving the efficiency of MAC in searching through highly constrained problems despite its high overhead, highlighted in Sabin and Freuder’s observation[8].

These results reveal the existence of a tradeoff between the computational overhead and the failure detection power and suggest important factors to be considered to choose an optimal approach to the given course scheduling problem: problem size and variable domain. The first important factor is the size of the dataset(i.e., number of courses to be scheduled). The dataset size is closely related to the constraint tightness and density of the problem, which serve as determinants of the MAC’s performance. Constraint density is inversely proportional to the number of possible constraints, which grows quadratically as the number of course variables increases. Constraint tightness generally goes up when the problem size gets larger, because the global size of the available time slots is fixed in a course scheduling problem.[8] MAC may be identified as an efficient approach if the problem size is above a certain threshold. The second factor is the size of the variable domain, or the bottleneck, to be specific. If the domain size of the bottleneck variables is already minimal (e.g., size one), the MRV heuristic can effectively resolve these constraints during the variable selection phase. In such cases, the additional domain pruning provided by MAC becomes redundant, as the potential search space reduction is already achieved through variable ordering, making a heuristic-only approach the most computationally efficient strategy.

7 Conclusion and Future Work

This study evaluated and compared the performance of three different backtracking configurations in the course scheduling problem, starting from the simple algorithm without heuristics and inference, to the rigorous algorithm with advanced techniques for search space optimization. The experiments were conducted over 4 datasets with varying levels of constraints, search space, and feasibility. The results suggest the following strategies to consider in tackling a course scheduling problem:

- For less-constrained, small-scale solvable course scheduling problems, basic backtracking remains a practical and efficient solution due to its low computational overhead.
- When the problem is more constrained, but the domain of the bottleneck node is small, MRV-only could be a suitable choice, as the domain-pruning technique becomes redundant considering AC-3’s high overhead.
- When the scheduling problem is large in size, highly constrained, or likely to be infeasible, MAC generally provides a clear advantage by detecting the contradiction through constraint propagation with minimal node exploration.

Despite the insights gained from this study, our study fails to fully capture the real-life course scheduling problem in multiple aspects. First, some of the major constraints, including the

section capacity, the distance between the adjacent courses' locations, and the courses with multiple required sections, were not considered, making the datasets less constrained than those in real life. Furthermore, the only heuristics used as a part of the experiments were the MRV and the degree heuristics, while the value ordering heuristic could have further reduced backtracks in solvable instances by selecting values for the variable in an optimal order, which might have improved the overall runtime of the backtracking algorithms. Lastly, the datasets constructed for the study lack practicality, assigning more than 10 courses to an individual, which implies an unrealistically heavy academic workload for a single student in a real-world scenario.

Future research will aim to narrow the gap between the simulation and practical applications by integrating complex constraints such as classroom capacities and geographical distances between course locations and evaluating various algorithms using more realistic, large-scale university datasets. Additionally, the value-ordering heuristics, specifically the Least-Constraining-Value approach, can be applied to determine their potential in diminishing the computational overhead of the MAC algorithm in solvable instances.

Team Members and Contributions

Jay Lee: Background (Propagation, CSP Application in Scheduling), Experiment Design, Results

Sanghwi Park: Background (Heuristics, Propagation), Analysis, Conclusion (Future work)

Jehoon Park: Abstract, Introduction, Background (Backtracking), Approach, Algorithms

References

- [1] Roman Barták, Miguel A. Salido, and Francesca Rossi. Constraint satisfaction techniques in planning and scheduling. *Journal of Intelligent Manufacturing*, 21(1):5–15, 2010.
- [2] James R Bitner and Edward M Reingold. Backtrack programming techniques. *Communications of the ACM*, 18(11):651–656, 1975.
- [3] Sally C. Brailsford, Chris N. Potts, and Barbara M. Smith. Constraint satisfaction problems: Algorithms and applications. *European Journal of Operational Research*, 119(3):557–581, 1999.
- [4] Michael W. Carter, Gilbert Laporte, and Sau Yan Lee. Examination timetabling: Algorithmic strategies and applications. *The Journal of the Operational Research Society*, 47(3):373–383, 1996.
- [5] Mark S. Fox and Norman M. Sadeh. Why is scheduling difficult? a CSP perspective. In *Proceedings of the 9th European Conference on Artificial Intelligence (ECAI)*, pages 754–767, 1990.
- [6] Robert M. Haralick and Gordon L. Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313, 1980.
- [7] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, 1977.
- [8] Daniel Sabin and Eugene C. Freuder. Contradicting conventional wisdom in constraint satisfaction. In Alan Borning, editor, *Principles and Practice of Constraint Programming*, pages 10–20, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
- [9] Edward Tsang. *Foundations of constraint satisfaction*. Books on Demand, Norderstedt, 1996.